Reading frontend design skill

Alright Ankit — this is Domain 12: **Frameworks, Tools & Ecosystem**. This is your practical arsenal — the actual libraries, platforms, and parsers that production RAG systems are built with. I'll build you a comprehensive interactive study guide covering all 8 sub-domains end-to-end.

Domain 12: Frameworks, Tools & Ecosystem

8 sub-domains · End-to-end coverage for senior GenAI interviews

LangChain | LlamaIndex | Haystack | Semantic Kernel | DSPy | Cloud RAG | Parsers | LLMs for RAG

DSPy — Declarative LM Programming

DSPy (Declarative Self-improving Python) is a paradigm shift from manual prompt engineering to **programmatic optimization**. Instead of writing prompts, you write *programs* that specify the input/output signature — DSPy then optimizes the prompts (and even few-shot examples) automatically to maximize a metric. Created at Stanford.

Stanford NLP | Prompt optimization | No manual prompting | Teleprompters / Optimizers

Core Philosophy: Programs over Prompts

Traditional Approach
Manually craft prompt strings. Brittle — breaks when model changes. Hard to optimize systematically. "Prompt engineering" as art.

DSPy Approach
Declare input/output signatures. Write a program using modules. Run an optimizer with a metric. Get optimized prompts automatically.

DSPy Modules

dspy.Predict — basic LM call

```
import dspy

# Configure the LM backend
lm = dspy.OpenAI(model="gpt-4o", max_tokens=500)
dspy.settings.configure(lm=lm)

# dspy.Predict: simplest module
# Signature format: "field1, field2 -> output1, output2"
classify = dspy.Predict("question -> answer, confidence")
result = classify(question="What is RAG?")
print(result.answer)    # "RAG stands for Retrieval Augmented Generation..."
print(result.confidence)  # "high"
```

dspy.ChainOfThought — structured reasoning

```
# ChainOfThought adds a "reasoning" field before the answer
cot = dspy.ChainOfThought("question -> answer")
result = cot(question="Is SunLife deductible higher than ManuLife?")
print(result.rationale)  # step-by-step reasoning
print(result.answer)     # final answer
```

dspy.RAG — built-in RAG module

```
class RAG(dspy.Module):
    def __init__(self, num_passages=3):
        super().__init__()
        self.retrieve = dspy.Retrieve(k=num_passages)
        self.generate = dspy.ChainOfThought("context, question -> answer")

    def forward(self, question):
        context  = self.retrieve(question).passages
        answer   = self.generate(context=context, question=question)
        return answer

rag = RAG(num_passages=5)
result = rag(question="What is ManuLife's dental coverage?")
```

Custom Signatures with typed fields

```
class InsuranceQA(dspy.Signature):
    """Answer insurance policy questions accurately and concisely."""
    question: str = dspy.InputField(desc="Insurance policy question from a customer")
    carrier:  str = dspy.InputField(desc="Insurance carrier name (e.g., SunLife)")
    answer:   str = dspy.OutputField(desc="Factual answer with relevant policy details")
    citations: list[str] = dspy.OutputField(desc="List of relevant policy sections")

qa_module = dspy.ChainOfThought(InsuranceQA)
```

Teleprompters (Optimizers)

Teleprompters are DSPy's optimization algorithms. They take your program + a metric + training examples and produce an *optimized* version of the program with better prompts and/or demonstrations.

BootstrapFewShot
Generates few-shot examples by running the program on training data, selecting examples where the metric is high.

BootstrapFewShotWithRandomSearch
Tries random subsets of bootstrap demonstrations to find the best few-shot composition.

MIPRO (Multi-prompt Instruction Proposal + optimization)
The most powerful optimizer. Uses Bayesian optimization over candidate instructions AND demonstrations. Best for production.

BayesianSignatureOptimizer
Optimizes instruction text using Bayesian search. Generates candidate instructions and scores them.

MIPRO Optimization Example

```
from dspy.teleprompt import MIPROv2

# Define your metric
def answer_accuracy(example, pred, trace=None):
    return example.answer.lower() in pred.answer.lower()

# Load training data
trainset = [
    dspy.Example(question="What is SunLife deductible?", answer="$500").with_inputs("question"),
    # ... more examples
]

# Compile (optimize) the program
optimizer = MIPROv2(metric=answer_accuracy, auto="medium")
optimized_rag = optimizer.compile(
    RAG(num_passages=5),
    trainset=trainset,
    max_bootstrapped_demos=3,
    max_labeled_demos=5,
    num_trials=15,
)

# The optimized_rag now has better prompts baked in
optimized_rag.save("optimized_insurance_rag.json")
```

> Why MIPRO matters in interviews: It separates *program structure* (what you code) from *prompt quality* (what DSPy optimizes). This is the future of "prompt engineering" — systematic, reproducible, metric-driven optimization instead of artisanal tweaking.

Systematic Prompt Optimization Flow

1. **Define Signature**
   Declare input fields, output fields, docstring (task description). No prompt text yet.
2. **Build Program**
   Compose modules (Predict, CoT, RAG, custom) into a forward() method.
3. **Define Metric**
   Write a function: (example, prediction) → float/bool. Could be exact match, LLM judge, F1.
4. **Compile (Optimize)**
   Run a Teleprompter. It generates demonstrations, proposes instruction variants, scores them against your metric.
5. **Evaluate & Iterate**
   Test on held-out set. If metric improves, deploy. Re-compile when data distribution shifts.

Here's your complete Domain 12 masterclass. Let me walk you through what's covered across all 8 tabs:

LangChain starts from LCEL fundamentals — the pipe `|` operator, `RunnableParallel`, `RunnablePassthrough` — and builds up through retrieval chains, conversational RAG with history-aware reformulation, all 6 advanced retriever types (MultiQuery, EnsembleRetriever, ParentDocumentRetriever, etc.), the text splitter hierarchy, and LangSmith integration.

LlamaIndex covers the three core index types and when to choose each, then digs deep into QueryEngine variants — especially `SubQuestionQueryEngine` (your multi-carrier comparison scenario) and `RouterQueryEngine`. The ingestion pipeline section shows how to use semantic chunking + metadata extractors + caching together.

Haystack explains the explicit component/socket pattern that differentiates it from LangChain, covers all DocumentStore backends, retriever types, and the built-in evaluation suite (ContextRelevance, Faithfulness, SAS evaluators).

Semantic Kernel maps the plugin → planner → memory architecture, shows Azure OpenAI wiring in Python, and has a C# vs Python comparison table.

DSPy is the conceptual heavy-hitter — covers the paradigm shift from prompt engineering to program optimization, all module types (Predict, CoT, RAG), and full MIPRO optimization flow with code. This section tends to be a strong interview differentiator since most candidates don't know it.

Cloud RAG has production-ready code for all 4 major platforms with concrete API examples — including your existing Azure AI Search hybrid retrieval pattern.

Parsers includes the full comparison matrix and production code for Docling, Unstructured, PyMuPDF, and Azure Document Intelligence — with a specific call-out to your insurance PDF pipeline use case.

LLMs for RAG gives you a decision framework by scenario — when to use full-doc Claude vs chunked GPT-4o vs local Llama 3, and why Command R+'s citation architecture is the answer to "how do you prevent hallucination."

Want me to do a focused deep-dive on any specific sub-domain, or should we move to mock interview Q&A for this domain?

Reply...

Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.